

ALASCA — Architecture logicielles avancées pour les systèmes cyber-physiques autonomiques

© **Jacques Malenfant**

Master informatique, spécialité STL – UFR 919 Ingénierie

Sorbonne Université
Jacques.Malenfant@lip6.fr

Cours 7

De l'entité adaptable — principes généraux et conception des capteurs

Plan

- 1 Principes de conception pour l'adaptabilité
- 2 Mesure, calcul et instrumentation
- 3 Conception et implantation des capteurs

Qu'est-ce qu'une entité adaptable dans un CPS ?

- Système cyber-physique :
 - Procédé : système *physique*, en fonctionnement *immersé* dans le monde réel (voiture autonome, drone, etc.).
 - Contrôleur : partie « cyber » pilotant le procédé.
- Lien entre procédé et contrôleur réalisé par des instruments cyber-physiques :
 - *Capteurs* : mesurent des propriétés sur le procédé et les rendent accessibles au contrôleur.
Ex.: capteur de température, GPS, tachymètre, etc.
 - *Actionneurs* : reçoivent des commandes informatiques et les transforment en actions sur le procédé.
Ex.: vanne, accélérateur, bras robotique, etc.
- Dans le contexte des systèmes cyber-physiques en général, nous avons adopté la terminologie :
 - Procédé : **Entité adaptable**, vue dans ce cours et le suivant (8).
 - Contrôleur : **Entité de contrôle**, vue aux cours 9 et 10.

Système autonome : éléments gérés

- Un système autonome est un CPCS¹ dont au moins une partie des entités adaptables sont des entités informatiques (programme, composants, intergiciels, etc.).
 - Adoptons la terminologie proposée par IBM :
entités adaptables informatiques = **éléments gérés**.
- Dans l'esprit des CPCS, les éléments gérés sont vus comme des procédés :
 - ils vont être dotés de « capteurs » et d'« actionneurs » ;
 - pour les contrôler, on pourra utiliser les *mêmes techniques* (automatique) que dans un CPCS classique.
- Ainsi, dans un système autonome, la partie cyber est constituée *à la fois* d'éléments gérés, et d'entités de contrôle.
 - La malléabilité des entités informatiques permet de réappliquer ce schéma *récurivement* (méta-récurivité).
 - Entités de contrôle adaptables *i.e.*, elles-mêmes adaptées par un *méta-contrôleur* et ainsi de suite (nous y reviendrons).

¹ Rappel : CPCS $\Rightarrow$ Cyber-Physical Control System.

Composants cyber-physiques : cahier des charges

- ❶ *Observation (monitoring)* : dotés d'opérations réalisant des mesures ainsi que d'interfaces capteurs, ports et connecteurs dédiés.
- ❷ *Adaptation* : dotés d'opérations d'adaptation ainsi que d'interfaces actionneurs, ports et connecteurs dédiés, les adaptations s'exécutant en exclusion mutuelle avec les services de base de l'entité.
- ❸ *Abstraction* : dotés d'interfaces *riches* exprimant les ressources requises et offertes par le composant et plus généralement l'ensemble de ses contraintes de fonctionnement et de contrôle.
- ❹ *Spécification* : intégrée au composant pour modéliser leur comportement à la fois par des modèles formels (automates hybrides) et exécutables (simulation).
- ❺ *Tests comportementaux* : embarquant une capacité de tests temporisés fondée sur les modèles (formels et exécutables).
- ❻ *Composition* : offrant des moyens théoriques et algorithmiques pour assurer leur composabilité par assemblage de composants pour former des applications globalement spécifiées et testables.

Propriétés d'adaptation : caractérisation

- Dans une vision théorie des systèmes, l'entité adaptable possède des *propriétés* qui décrivent ses entrées/sorties, son état interne, ses qualités de service, etc.
- Typologie de propriétés d'adaptation en pratique :
 - *Modifiables* : entrées directement modifiables en changeant leur valeur (ex.: nombre de MV allouées).
 - *Influençables* : sorties *indirectement* modifiables en agissant sur des propriétés modifiables ; ex.: temps moyen de réponse aux requêtes influencé par le nombre de MV,
 - *Contingentes* : entrées non maîtrisées, typiquement de l'environnement ; ex.: taux d'arrivée des requêtes dans une application web.
- Rôle des propriétés d'adaptation dans l'élément géré :
 - *Fonctionnelles* : jouent un rôle direct dans son exécution (ex.: taille bornée de la file d'attente des requêtes).
 - *Non-fonctionnelles* : propriétés simplement observées, ne jouant pas de rôle dans l'élément (ex.: temps moyen de réponse).

Éléments de conception pour l'adaptabilité

- 1 **Conception orientée propriétés d'adaptation** : quelles sont ces propriétés, leurs caractéristiques, les actions ou opérations permettant de les modifier ou influencer ?
- 2 **Contrôles** : quels sont les contrôles envisagés et leurs objectifs ? quelles sont les propriétés impliquées et les propriétés influençables sur lesquelles des objectifs sont fixés ?
- 3 **Mesurabilité** (influençables) : comment leurs valeurs seront obtenues ? par capteurs cyber-physiques ? par mesures simplement collectées ? par des calculs complexes ? avec quelles méta-propriétés ?
- 4 **Visibilité** (influençables et contingentes) : de quelle façon les valeurs de ces propriétés sont rendues visibles de l'extérieur de l'entité ? avec quels moyens de configuration ?
- 5 **Contrôlabilité** (influençables) : les opérations envisagées permettent-elles de modifier ou d'influencer les propriétés de manière suffisamment précise pour d'atteindre les objectifs fixés pour les contrôles ?

Conception orientée propriétés d'adaptation

Conception orientée propriétés d'adaptation

*Toute entité adaptable doit être conçue autour d'un ensemble de **propriétés d'adaptation**, de leurs relations avec les **opérations de mesures et d'adaptation** ainsi qu'avec les **contrôles** qui sont envisagés.*

- Difficultés potentielles par rapport aux contrôles envisagés :
 - Assurer la mesurabilité et l'observabilité de toutes les propriétés utilisées, avec les qualités nécessaires.
 - Assurer l'existence des opérations d'adaptation couvrant toutes les propriétés influençables identifiées.
 - Assurer la faisabilité des contrôles envisagés.Les objectifs doivent être atteignables avec les moyens réalisables en pratique.

Mesurabilité

Principe de mesurabilité

*Toute propriété influençable ou contingente utilisée dans un contrôle doit être **mesurable**, via une instrumentation disponible, des **mesures directes** ou des **calculs** réalisables.*

- Mesurabilité : capacité à connaître les valeurs des propriétés *i.e.*, collecter des mesures et calculer ces valeurs.
 - Les propriétés modifiables sont trivialement mesurables puisque par définition on les choisit. . .
 - Pour les autres, c'est plus complexe ; on peut distinguer :
 - la faisabilité de l'opération de mesure, qui peut se révéler impossible en pratique ;
 - la précision de la mesure et ses autres méta-propriétés, qui peuvent être insuffisantes pour les contrôles envisagés, et
 - le coût de cette opération, qui peut rendre le système trop inefficace ou trop coûteux à réaliser.

Chacun de ces points peut rendre les contrôles à exercer sur ces propriétés impossibles à réaliser en pratique.

Visibilité : une notion logicielle

Principe de visibilité

*Les opérations de mesure et d'adaptation sont rendues **visibles** aux entités tierces à travers un ensemble de services déclarés par ses **interfaces** capteurs et actionneurs.*

- Les interfaces capteurs représentent le seul moyen pour une autre entité (contrôleur) d'obtenir les valeurs des propriétés adaptables de l'entité adaptable.
 - Distinguons bien la *mesure* et *lecture* car plusieurs lectures peuvent utiliser la même mesure entre deux mesures successives.
- Les interfaces actionneurs représentent le seul moyen pour une autre entité (contrôleur) de modifier (contrôler ou influencer) les valeurs des propriétés adaptables de l'entité adaptable.
- Ces services ont aussi des *qualités* ou *méta-propriétés* à éliciter et, potentiellement, à configurer.

Contrôles et contrôlabilité

- Expliciter les propriétés adaptables, leur mesurabilité, leur visibilité est central, mais insuffisant pour assurer la faisabilité des contrôles envisagés.
- Il faut aussi s'assurer de la capacité des opérations d'adaptation à influencer les valeurs des propriétés concernées de manière à *vraiment atteindre* l'ensemble-cible dans un *délaï borné*.

Principe de contrôlabilité

Toute **propriété influençable** faisant l'objet d'un contrôle doit être influencée par un ensemble d'**opérations** d'adaptation **visibles**, dont les **effets** sur cette propriété sont **mesurables** et **visibles**, doit également garantir qu'à partir de toute valeur initiale admissible, ces opérations peuvent la ramener **aussi près que souhaité** de l'objectif du contrôle dans un **temps bornable**.

Plan

- 1 Principes de conception pour l'adaptabilité
- 2 Mesure, calcul et instrumentation**
- 3 Conception et implantation des capteurs

Fondements

Mesurer (source Wikipedia, 25/09/2014)

En physique et en sciences de l'ingénieur, mesurer consiste à comparer une grandeur physique qui caractérise un objet (ou un événement) avec celle de même nature choisie comme unité de mesure. La valeur numérique (*sic*¹) de la grandeur mesurée est le nombre (*sic*) qui fixe la relation entre la grandeur mesurée et l'unité de mesure choisie. Le dispositif qui permet de réaliser la mesure est un instrument de mesure (ou appareil de mesure).

Instrumentation (source Wikipedia, 25/09/2014)

En sciences, l'instrumentation est une technique de mise en œuvre d'instruments de mesure, d'actionneurs, de capteurs, de régulateurs, en vue de créer un système d'acquisition de données ou de commande.

Nota : instrumentation = mesure (capteurs) et adaptation (actionneurs).

¹ Il n'y a pas de raison de se limiter à des valeurs *numériques* ! On peut aussi utiliser des valeurs qualitatives, comme en logique floue, ou encore des valeurs structurées, comme en réflexion.

Cycle de production en instrumentation et mesure

- Étapes standards :
 - 1 Grandeur physique : propriété à mesurer et ses caractéristiques intrinsèques (type, unité de mesure, etc.).
 - 2 Mesure : procédé par lequel la grandeur physique va être traduite en donnée utilisable.
 - 3 Conditionnement : dans la durée, conversion des mesures en un signal reflétant l'évolution temporelle de la grandeur physique, tout en éliminant les biais systématiques dûs à la technique de mesure.
 - 4 Traitement du signal : élimination des erreurs statistiques de mesure par filtrage.
 - 5 Exploitation : consommation des données produites.
- *Si ce cycle de production est proposé en physique et électronique pour les systèmes cyber-physiques, ces différents concepts sont également pertinents pour l'informatique autonome et leurs éléments gérés !*

Propriétés et mesure

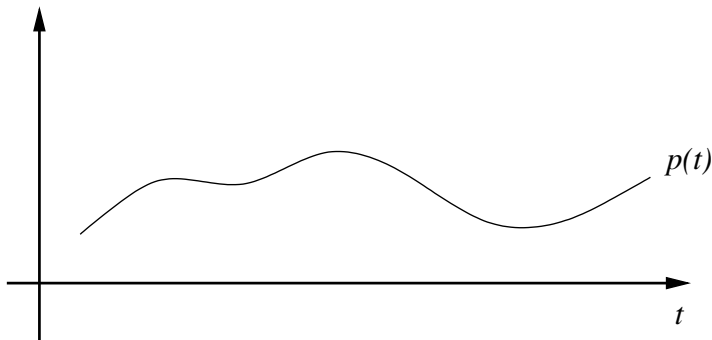
- Pour les éléments gérés, la mesure se réalise souvent par programmation, ce qui implique :
 - de programmer le code réalisant la collecte des données brutes et le calcul des valeurs de propriétés ;
 - d'exécuter ce code répétitivement, à des instants *discrets*, c'est-à-dire procéder à un *échantillonnage*.
- Toute mesure implique :
 - un certain *effort* ou *coût* et un délai pour l'obtenir,
 - une certaine *imprécision* ou *erreur*, et
 - une certaine *perturbation* du système (effet « quantique »).
- Mesurer suppose donc un compromis entre *ressources consommées*, *perturbation* du système et *qualités* de la mesure.
 - La mesure n'est pas « masquable » à l'intérieur d'une boîte noire car le *comment* est tout aussi important que le *quoi*.
 - Les *méta-propriétés* des opérations de mesure expriment sa qualité et doivent être documentées par l'interface capteur.
 - Le compromis dépendant des propriétés du contrôle à exercer, la *configurabilité* des algorithmes de mesure est *essentielle*.

Évolution et mesure temporelle : l'échantillonnage

- Une propriété p dont la valeur évolue dans le temps peut être modélisée par une fonction $p(t)$.
- Rappel : en informatique, toute mesure suppose de l'*échantillonnage* discret pour détecter les changements de valeurs et mesurer ces dernières.
- Échantillonner se décline à deux niveaux :
 - détecter, mesurer ou calculer des valeurs à des instants discrets *i.e.*, *échantillonner pour mesurer* une propriété donnée grâce à l'algorithme implantant le capteur ;
 - lire les valeurs successives à des instants discrets dans le temps *i.e.*, *échantillonner pour surveiller* la propriété via l'interface capteur la rendant visible.
- Mesure et lecture étant indépendantes l'une de l'autre, ces échantillonnages et leurs interactions ont des impacts significatifs sur les contrôles à exercer.

Exemple : évolution d'une propriété p

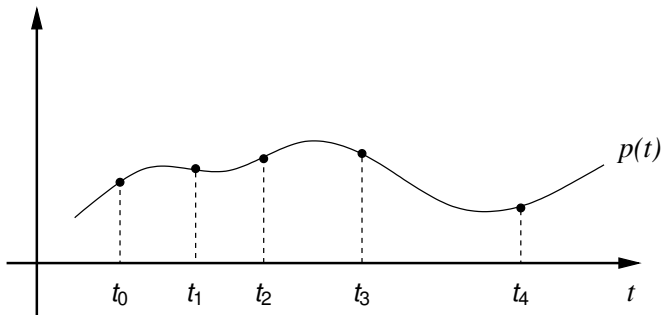
- Soit une propriété p évoluant dans le temps représentée par une fonction $p(t)$:



- Cette fonction pourrait résulter d'une mesure *analogique* continue (ex.: signal électrique sous oscilloscope).
- En informatique, elle va devoir être *échantillonnée* à des instants discrets.

Résultat de l'échantillonnage

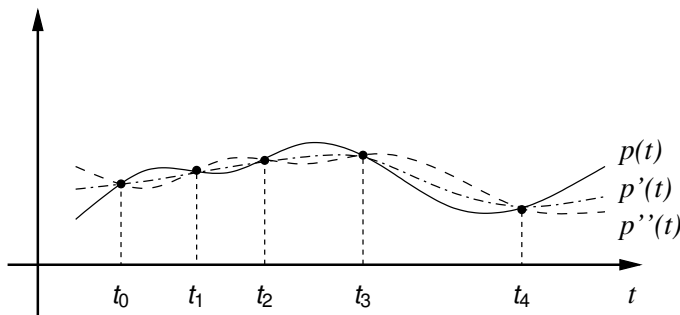
- Mesurée ou lue à des instants t_0, t_1, t_2, t_3, t_4 , ce qui donne des valeurs $p[t_0], p[t_1], p[t_2], p[t_3], p[t_4]$:



La courbe $p(t)$ est illustrée comme visualisation abstraite d'une variation **possible** de p ; en fait, **seules les valeurs $p[t_i]$** sont réellement connues, par la mesure ou la lecture (selon le cas étudié), et encore sont-elles entachées d'une erreur de mesure.

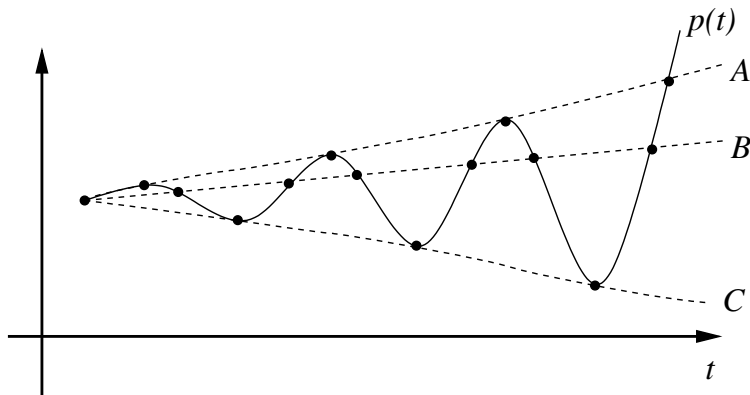
Problème engendré : aliasage

- Problème de l'échantillonnage : manque d'information permettant d'identifier de manière *unique* la fonction $p(t)$ à partir des seuls échantillons $p[t_0]$, $p[t_1]$, $p[t_2]$, $p[t_3]$, $p[t_4]$:



- On voit ici qu'avec les *mêmes échantillons*, nous aurions pu *aussi légitimement déduire* les fonctions $p'(t)$ ou $p''(t)$.
Laquelle est la bonne ?

Est-ce grave docteur ?



- *A : ça croît bien !*
- *B : ça croît, mais lentement !*
- *C : ça décroît bien !*

- *En plus, il existe une infinité de fonctions alias de $p(t)$ se confondant exactement avec elle aux points d'échantillonnage mais différant partout ailleurs.*

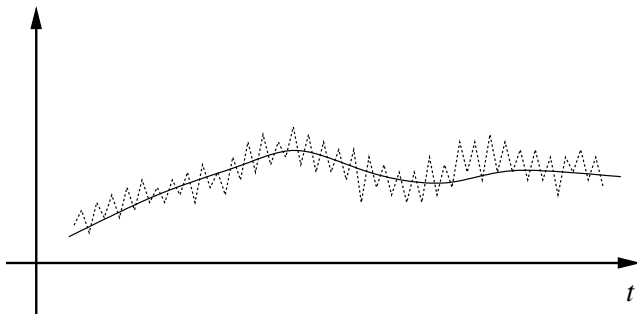
Aliasage et contrôle

- Quelle $p(t)$ étant donnés les $p[t_i]$?
 - En contrôle, différents alias $\Rightarrow$ différentes décisions ;
ex.: $p(t)$ paraît croissante à t_0 alors que $p''(t)$ paraît décroissante.
 - On suppose généralement que la vraie fonction épouse de près l'interpolation linéaire par morceaux, mais méfiance. . .
- **Aucune** fréquence ou séquence d'échantillons permet de garantir l'absence d'aliasage ! De plus, attention :
 - Sous-échantillonnage : des variations transitoires mais importantes peuvent passer inaperçues au contrôleur.
 - Sur-échantillonnage : augmentation du coût et risque de faire apparaître de variations non significatives dues aux erreurs de mesure.
- *Comment déterminer une fréquence d'échantillonnage appropriée ?*
 - Bien étudier au préalable, sur le terrain, le phénomène échantillonné, son amplitude, son évolution.
 - À chaque fréquence choisie correspond une limite sur la variation de la fonction et ses dérivées qui pourra être capturée.

Problématique de l'erreur de mesure et le filtrage

- Toute mesure est entachée d'erreur, donc suivre les variations de trop près peut induire le contrôleur en erreur.
- Réduire les conséquences de ces erreurs est l'un des thèmes centraux du domaine dans lequel on parle de *bruit*.
- En réalité, on constate deux formes de bruit :
 - le bruit du processus lui-même qui peut avoir une part d'évolution aléatoire intrinsèque, et
 - le bruit de mesure, lié aux instruments ou procédés d'observation employés.
- Le terme filtrage est généralement employé pour désigner l'ensemble des procédés développés pour éliminer le bruit, qu'il provienne du processus ou de la mesure.

Illustration du phénomène



- En pointillés, $p'(t)$ obtenue en reliant par des segments de droites les mesures *bruitées*.
 - Mène potentiellement à surréagir à des variations insignifiantes.
- En trait plein, $p''(t)$ la fonction obtenue après filtrage.
 - Cherche à faire apparaître uniquement les *évolutions de fond* auxquelles le contrôle doit s'appliquer.

Comment filtrer ?

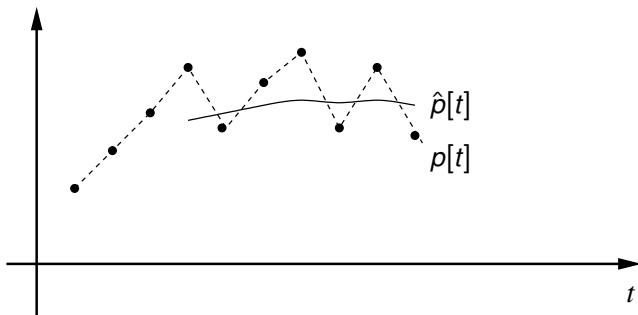
- Idée de base : utiliser plusieurs mesures conjointement de manière à réduire les variations dues au bruit.
- Exemple : moyenne des n dernières mesures
 - Soient $p[i], i = 0, 1, \dots$ les mesures bruitées.
 - Produire la séquence filtrée $\hat{p}[i] = \frac{\sum_{j=i-n+1}^i p[j]}{n}, i \geq n$
- C'est ce qu'on appelle un filtre à moyenne mouvante.
- Exemple : pour $n = 4$

t	0	1	2	3	4	5	6	7	8	9
$p[t]$	1.0	1.3	1.5	1.8	1.4	1.7	1.9	1.4	1.8	1.3
$\hat{p}[t]$				1.4	1.5	1.6	1.7	1.6	1.7	1.6

Nota : la moyenne ne peut être calculée avant d'avoir au moins n mesures ; il faut une alternative au début du processus.

Comparatif $p[t]$ versus $\hat{p}[t]$

- Sur les données de l'exemple précédent :

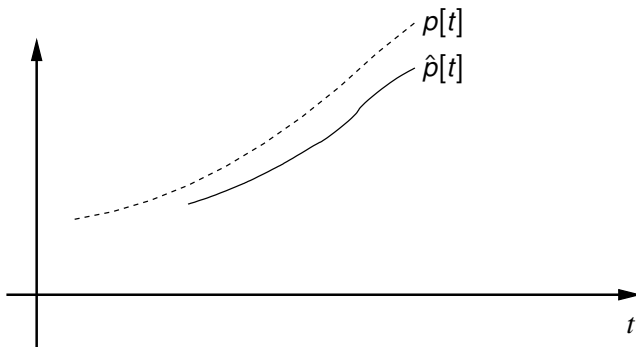


- On constate que $\hat{p}[t]$ lisse bien les mesures bruitées et produit une série de mesures filtrées dégagée des variations aléatoires autour d'une évolution générale jugée fondamentale.

Problème potentiel : inertie du filtrage

- Nouvelles données :

t	0	1	2	3	4	5	6	7	8	9
$p[t]$	1.0	1.2	1.5	1.9	2.4	3.0	3.7	4.5	5.4	6.4
$\hat{p}[t]$				1.4	1.75	2.2	2.75	3.4	4.15	5.0



- Dans cet exemple, les mesures filtrées retardent systématiquement sur les mesures brutes, mais on pourrait observer toutes sortes d'autres distorsions (déphasages, ...).

Filtres plus performants

- Il existe d'autres filtres que la moyenne mouvante, potentiellement meilleurs ou plus souples.
- Filtre *exponentiel* : avec $0 < \alpha \leq 1.0$

$$\hat{p}[j] = \alpha p[j] + (1 - \alpha) \hat{p}[j - 1]$$

- Par α , on règle les poids respectifs des nouvelles valeurs par rapport aux anciennes.
 - $\alpha \rightarrow 1.0$ si on veut suivre et réagir aux évolutions du plus près, et
 - $\alpha \rightarrow 0.0$ si on veut limiter les effets du bruit ponctuel.
- Filtres à *prédiction linéaire* : les valeurs précédentes sont utilisées pour calculer la dérivée (supposée moins variable) à partir de laquelle on prédit la valeur courante, prédiction qui est ensuite combinée à la mesure courante pour produire une valeur filtrée.
- Filtre de Kalman : filtre optimal sur prédiction linéaire en présence de bruit « blanc ».

Cohérence des mesures utilisées conjointement

- Abadi et Lamport ont montré qu'une inférence (calcul) à partir d'un ensemble de mesures n'est *bien fondée* que si elle utilise des mesures *cohérentes i.e.*, issue d'un *même état*.
- En pratique, cela veut dire qu'il faut que ces mesures soient prises à des instants *suffisamment proches*.
- Trois problèmes :
 - ① retrouver le moment de prise des mesures par datation,
 - ② découpler lecture et mesure pour permettre de planifier les mesures afin de construire des ensembles cohérents et
 - ③ assurer des moyens de lire ces mesures qui permettent de récupérer des ensembles cohérents.
 - Prévoir des capteurs (logiques) retournant toutes les valeurs nécessaires en même temps *i.e.*, *jointure temporelle côté mesure*.
 - Permettre de configurer les fréquences de mesure des propriétés nécessaires que le client puisse créer lui-même des ensembles de valeurs cohérents malgré leur lecture séparée *i.e.*, *jointure temporelle côté client*.

L'introspection réflexive vue comme mesure

- Un autre lien nécessaire doit être fait entre mesure et réflexion.
- L'introspection consiste à obtenir des informations sur l'état d'une entité pendant son exécution, ce qui peut s'apparenter à une mesure.
- Par rapport à la mesure de propriétés de qualité de service, il s'agit ici le plus souvent de propriétés discrètes, dont l'évolution se fait ponctuellement à des instants discrets.
- La fréquence de ces modifications varie de peu fréquent pour les informations structurelles (code, architecture, etc.) à relativement fréquent s'il s'agit d'informations sur l'état d'exécution (état de la pile d'exécution, mémoire consommée, liens entre objets, etc.).
- Ici, la durée d'exécution du code de « mesure » peut facilement devenir assez importante (ex.: récupérer la forme du graphe d'objets en mémoire), et donc doit être prise en compte dans l'utilisation de ce type de mesures.

Spécification complète : méta-propriétés des mesures

- Quelques méta-propriétés :
 - Domaine de définition.
 - Taux de variation attendu, éventuellement en fonction d'autres propriétés lorsqu'une relation existe entre elles.
 - Fréquence maximale et inertie imputable au procédé de mesure employé.
 - Précision, biais systématique éventuel, densité de probabilité de l'erreur aléatoire.

Principe : méta-propriétés explicites des mesures

*Toute mesure, réalisée de manière endogène par une entité ou exogène par des instruments externes, doit être **spécifiée** par des **méta-propriétés explicites**.*

- *Interfaces riches* : explicitations des méta-propriétés au minimum par de la documentation textuelle ou des annotations sur les interfaces capteurs des entités adaptables.

Plan

- 1 Principes de conception pour l'adaptabilité
- 2 Mesure, calcul et instrumentation
- 3 Conception et implantation des capteurs**

De la mesure aux capteurs

- Rappels :
 - Dans un CPCS classique, c'est un instrument avec :
 - un dispositif matériel assurant la mesure ;
 - une « interface » (pilote) permettant la lecture des données par informatique.
 - Par analogie, dans un système autonome, un capteur a :
 - une partie logicielle intégrée à l'élément géré assurant la mesure et la mise en forme des données ;
 - une interface (signatures) permettant la lecture des données.
- Organisation de la partie interface :
 - Au-delà des signatures des opérations de lecture, l'interface capteur doit expliciter les méta-propriétés des données.
 - Les méta-propriétés peuvent faire l'objet d'une surveillance à l'exécution, en offrant des opérations dédiées (mesure sur la mesure).
 - Les algorithmes de mesure peuvent être paramétrés pour expliciter les compromis d'implantation et permettre d'agir sur ces choix par des opérations de configuration.

Conception de la fonction capteur

- Objectif : conception des dispositifs organisant les mesures et donnant accès aux valeurs.
- Problématique : proposer un ensemble de capteurs dont les interfaces soient flexibles, ouvertes et robustes, permettant à des entités tierces d'observer pour adapter l'entité à bon escient.
- Éléments de conception et mise en œuvre à couvrir :
 - déclinaison des exigences pour la fonction capteurs ;
 - définition d'un modèle pour les données de capteurs ;
 - mise en œuvre des mesures *ad hoc* et injection du code correspondant au sein des éléments gérés ;
 - définition d'interfaces flexibles et ouvertes pour l'émission et la transmission des données (issues des mesures) ;
 - intégration d'une capacité de configuration et d'adaptation dynamique de la fonction capteurs ;
 - explicitation dans ses interfaces des informations nécessaires pour utiliser correctement des capteurs.

Qualités souhaitées d'un ensemble de capteurs

Simplicité : produire l'ensemble des données utiles uniquement par les moyens *nécessaires et suffisants*.

Modularité : *découpler* les différents capteurs autant que possible pour faciliter la gestion de leurs cycles de vie respectifs (conception, implantation, tests, mises à jour, etc.).

Configurabilité : permettre de configurer les paramètres d'exécution comme les fréquences de mesure.

Redondance : introduire suffisamment de *redondance* entre capteurs pour induire *tolérance aux pannes et robustesse* (surtout capteurs liés aux phénomènes physiques).

Méta-propriétés explicites : déduire, exprimer et déclarer *explicitement* les méta-propriétés de chacun des capteurs.

Auto-évaluation : offrir une capacité réflexive pour *superviser* la qualité de service et les méta-propriétés des capteurs (« capteurs de capteurs »).

Modèle générique de données mesurées

- Concevoir une instrumentation correcte et efficace exige la définition du modèle des données de mesure elles-mêmes.
- Une mesure se caractérise d'abord par :
 - sa *valeur*, avec toutes les méta-propriétés identifiées (domaine, précision, erreur moyenne, biais, ...);
 - l'*unité de mesure*, sans laquelle la valeur ne peut pas être interprétée utilement (standards, JSR-275, JSR-363);
 - la *date* (heure) de la mesure, qui permettra à la fois de raisonner sur elle en tenant compte de son âge et de vérifier la cohérence temporelle des données utilisées conjointement;
 - l'*identité* du dateur, par exemple l'adresse IP de l'hôte possédant l'horloge ayant servi à la datation, ce qui permettra éventuellement d'interpréter correctement la date.
- Toute mesure produite, transmise et consommée doit être définie par ces quatre éléments d'information.

Mesures fournies par des instruments existants

- Instrumentation déjà implantée par des outils existants :
 - Outils mesurant la consommation des ressources du système et de l'ordinateur physique (ex.: `ps` ou `top` sous Unix).
 - Services de métrologie offerts par le réseau, pour tout ce qui concerne les échanges de données (ex.: commande `airport`).
 - Services spécifiques offerts par différentes applications (bases de données, serveurs Web comme Apache, etc.).
- Leur utilisation pour la mesure d'une propriété demande :
 - ① identifier parmi les mesures offertes celles qui sont pertinentes,
 - ② avoir une portée adéquate permettant d'associer une mesure générale à la propriété de l'entité visée,
 - ③ exhiber les bonnes méta-propriétés (précision, biais, ...), et
 - ④ être capable d'extraire ces données *efficacement*.
- Si ces mesures peuvent paraître *gratuites*, ne jamais oublier le coût à l'exécution (5-6% du CPU pour `top`, par exemple, selon les paramètres donnés au lancement).

Développement de la fonction capteur

- ❶ Concevoir les procédés et algorithmes de mesure.
- ❷ Identifier les données brutes nécessaires pour calculer la(les) mesure(s).
- ❸ Injecter du code pour *récolter les données brutes* aux endroits et aux moments où elles sont disponibles.
 - Portée spatiale : portion de code sur laquelle porte la mesure.
 - Portée temporelle : durée sur laquelle porte la mesure.
- ❹ Injecter du code pour le *calcul* des mesures à partir des données brutes et en adéquation avec leur récolte (*i.e.*, à quels moments ?).
- ❺ Tester, estimer les méta-propriétés et valider les instruments de mesure sur des cas réalistes.
- ❻ Identifier les sources éventuelles de biais et introduire des compensations dans le code de calcul des mesures.
- ❼ Étudier les propriétés de l'évolution des mesures et leurs erreurs systématiques dans le temps et, si nécessaire, ajouter du filtrage.

Injection du code de mesure

- Ayant identifié les mesures à réaliser et conçu les algorithmes de mesure appropriés, deux possibilités s'offrent au développeur :
 - une approche manuelle, par laquelle le développeur introduit lui-même le code nécessaire en chacun des points du code source où les algorithmes exigent d'intervenir ou
 - une approche plus systématique et mieux organisée, par laquelle le développeur va pouvoir
 - définir en un seul endroit le code nécessaire pour la mesure et
 - utiliser un moyen automatique pour injecter les différentes parties de ce code aux différents endroits nécessaires dans le source.
- Méthodes d'injection automatique
 - Approche réflexive : interception (*proxy*, méta-objets) ou insertion réflexive de code à la volée.
 - Approche par aspects : définition modulaire de code à injecter et de points de jointures dynamiques.
⇒ programmation par aspects, AspectJ.

Organisation du processus de prise des mesures

- Question : comment ordonnancer la prise (calcul) des mesures ?
 - Plusieurs considérations :
 - disponibilités des données brutes : où et quand ?
 - capteurs mono/multi-mesures *i.e.*, cohérence temporelle,
 - besoin des clients *i.e.*, fréquence des lectures, jointures temporelles, filtrage, méta-propriétés, etc.
 - Faire de *nouvelles mesures à chaque lecture* n'est pas toujours possible :
 - mesures peuvent devoir se faire à un autre moment/endroit,
 - latence de mesure peut trop retarder les lectures,
 - lectures trop fréquentes pour mesurer à chaque fois, etc.
- ⇒ planification des mesures indépendantes des lectures.
- Fréquences dépendent des besoins des clients car :
 - crée deux niveaux d'échantillonnage (mesure et lecture), source supplémentaire d'erreurs ;
 - doit éviter le sur-échantillonnage ou le sous-échantillonnage.
- ⇒ moyens de configurer en fonction des besoins des clients.

De la mesure au capteur et à son interface

- Une fois les algorithmes de mesures déterminés et réalisés, il faut ensuite expliciter les méta-propriétés de ces mesures et les documenter sur leurs interfaces dites riches.
Ex.: documentation Javadoc, mais aussi utilisation des annotations Java pour capturer les méta-propriétés.
- Ensuite, il faut passer à l'exposition des résultats de ces mesures à travers les interfaces capteurs.
- Pour chaque capteur rendu visible, il faut déterminer deux mécanismes centraux :
 - 1 le mode d'émission des données, et
 - 2 le mode de transmission des nouvelles données.
- Ces mécanismes peuvent mettre en jeu des inter-relations entre capteurs (mesures communes, par exemple) nécessitant la possibilité de les définir conjointement.

Mode d'émission des mesures via les capteurs

- Deux modes temporels principaux :
 - ① émission passive (*pull*) : le consommateur de la mesure la demande en appelant l'entité sur l'interface capteur.
 - ② émission active (*push*) : le consommateur s'abonne au capteur et ce dernier lui envoie les mesures selon un certain programme de publication, le plus souvent à une certaine fréquence fixe.
- Principales propriétés de configuration déterminant le fonctionnement capteur/mesure et capteur/lecture :
 - Dans le mode passif, il faut déterminer quand un appel doit donner lieu à une nouvelle mesure, si la mesure se fait au besoin.
 - Dans le mode actif, la fréquence des mesures peut être fonction des fréquences d'émission et de la fraîcheur requises par les clients.
- Attention : un capteur peut être utilisé par *plusieurs* clients avec des besoins différents (mode, paramètres de fonctionnement).
Principe : produire une fois, utiliser plusieurs fois !

Mode de transmission des nouvelles mesures

- Quoi transmettre à chaque émission :
 - 1 La nouvelle mesure telle que produite ?
 - 2 Seulement les différences par rapport à la dernière valeur ?
- Deux modes de transmission essentiellement.
 - 1 Par rafraichissement complet :
 - transmission complète de la nouvelle mesure ;
 - s'applique bien aux données de petite taille.
 - 2 Par mise-à-jour différentielle :
 - transmission des modifications ou du delta par rapport à l'ancienne mesure (genre `diff/patch` sous Unix) ;
 - s'applique bien aux données de grande taille mais pour lesquelles les évolutions sont localisées,
 - mais moins robuste aux pertes !
 - ⇒ *ajouter un historique des dernières modifications à chaque transmission ?*
 - ⇒ *adopter un mode mixte ?*
- Compromis nécessaire entre efficacité et robustesse.

Configuration des émissions et des mesures I

- Prenons un scénario concret pour comprendre :
 - Deux mesures : CPU et MEM,
 - deux capteurs en mode *push*,
 - utilisées par trois contrôleurs :
 - 1 Le contrôleur 1 veut CPU toutes les 60 secondes avec une fraîcheur d'au plus 10 secondes.
 - 2 Le contrôle 2 veut MEM toutes les 30 secondes aussi avec une fraîcheur d'au plus 10 secondes.
 - 3 Le contrôle 3 veut MEM et CPU toutes les 40 secondes en cohérence temporelle de moins de 20 secondes avec une fraîcheur d'au plus 30 secondes pour les deux données.
- Problème : fixer les instants de mesure et d'émission des deux propriétés permettant de satisfaire ces besoins.
- Hypothèse : démarrage synchronisé de la production

Configuration des émissions et des mesures II

- PPCM des fréquences de production est 120 secondes, donc un programme sur cette durée pourra être répété indéfiniment.
- Déroulement possible :
 - ❶ 0s : mesures CPU et MEM, puis envoi à 1, 2 et 3.
 - ❷ 30s : nouvelle mesure MEM puis envoi à 2.
 - ❸ 40s : nouvelle mesure CPU (fraîcheur = 40s > 30s et cohérence = 40s > 20s) et lecture de MEM puis envoi de CPU+MEM à 3.
 - ❹ 60s : nouvelles mesures CPU et MEM (fraîcheur = 20s et 30s > 10s) puis envois à 1 et 2.
 - ❺ 80s : lecture et envoi CPU+MEM à 3.
 - ❻ 90s : nouvelle mesure MEM (car fraîcheur = 30s > 10s) puis envoi à 2.
 - ❼ 120s : nouvelles mesures CPU et MEM (fraîcheur CPU = 40s > 30s et alors cohérence = 30s > 20s), puis envois à 1, 2 et 3.

et on boucle...

Interface de configuration

- Comment généraliser cet exemple et définir de manière la plus générique possible une interface de configuration ?
 - interface opérationnelle : fixe pour chacune des propriétés le programme précis de ses mesures ;
 - interface déclarative : exprime les besoins sous forme de contraintes et calcule un programme pour toutes les mesures.
 - Pas besoin de connaître les détails internes de mise en œuvre ;
 - permet l'auto-configuration et facilite l'adaptation dynamique.
- Vers les systèmes *Complex Event Processing* (CEP, §Cours 14) :
 - Ensemble de concepts, de techniques et d'outils permettant de comprendre, analyser et contrôler les systèmes d'information dirigés par événements.
 - Systèmes répartis de détection, de propagation causale, de corrélation et de traitement à grande échelle des événements.

Récapitulons...

- 1 L'entité adaptable est le **parent pauvre** de l'informatique autonome alors que ses **fonctions capteurs et actionneurs** et son comportement sous adaptation sont pourtant la **pièce angulaire** d'un système auto-adaptable.
- 2 Une entité adaptable doit être conçue autour d'un **modèle de comportement**, intégrant ses opérations d'adaptation et ses modes de fonctionnement.
- 3 La **mesure** de propriétés est un processus qui peut être coûteux et qui peut perturber de manière importante l'entité mesurée.
- 4 De plus, sur les systèmes informatiques, les propriétés continues ne peuvent être mesurées que par des **interventions discrètes**, ce qui soulèvent des difficultés d'**échantillonnage**.
- 5 Les **erreurs**, inhérentes la mesure, doivent être appréhendées explicitement, et l'utilisation de **filtres**, malgré leurs inconvénients, est souvent une nécessité.
- 6 La mise en œuvre des capteurs nécessitent la réutilisation ou la mise en place de **mesures**, le choix de modes d'**émission** et de **transmission** des données, et la possibilité de **configurer** les interfaces pour servir plusieurs clients.
- 7 Du besoin d'utiliser **conjointement** de nombreuses mesures capteurs découle la nécessité de les **synchroniser**, soit par des opérations de **mesures jointes** ou par de la **datation systématique**.

Pour aller plus loin : sélection de lectures recommandées

- *An architectural blueprint for autonomic computing*, IBM, 4^e édition, 2006.
- *Autonomic Computing*, P. Lalanda, J. McCann et A. Diaconescu, Springer-Verlag, 2013, chapitre 4.
- *Stochastic models, estimation and control volume 1*, P.S. Maybeck, Academic Press, 1979, chapitre 1.